ATLAS

Distributed Object-Tolerant Storage Framework

Distributed Systems | Universite Antonine | 28 October 2026

Implementation, demonstration guide, verification evidence and outstanding checks. This report separates working demonstrations from prepared bonus deployment files.

Part	Implementation and evidence
1. Environment	Python virtual environment, Git remote, Docker and Kubernetes clients configured.
2. Storage nodes	Eight independent persistent storage nodes; SHA-256 checks and heartbeats.
3. Metadata and placement	Fixed 4-MiB chunks, optional compression, versions, virtual-node consistent hashing and reference-counted deduplication.
4. Redundancy and recovery	N=3,W=2,R=2 replication; per-bucket zfec 4+2 coding; repair and scrub workers. Small-workload repair measured. Final failure-suite status must be checked in recorded JSON.
5. Data API	Streaming/multipart upload, byte ranges, metadata, search, API keys and shared rate limiting. Initial live API run: 10 passed; search was deferred during image download.
6. React UI	Built and opened at localhost:8088. A real file upload was observed. Screenshot evidence follows.
7-8. Bonus	Dockerfiles, worker queues, monitoring, Kubernetes manifests and GitHub workflows prepared. Complete remote deployment is not claimed.

Current status and exact commands

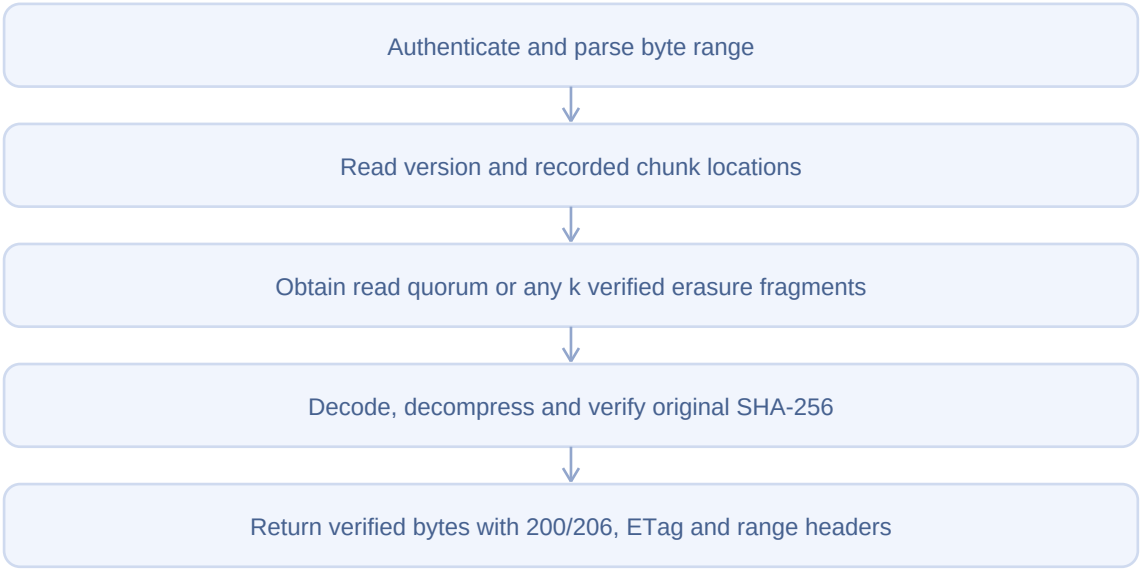
Read docs/RUN-AND-DEMONSTRATE.md for the current acceptance notes, credentials location, workload scope and demonstration commands. This report intentionally contains no private passwords.

Write path



A bucket selects replication or erasure coding. The content hash identifies original bytes; fragment hashes identify encoded stored bytes. Writes become visible only after their metadata transaction commits.

Read path

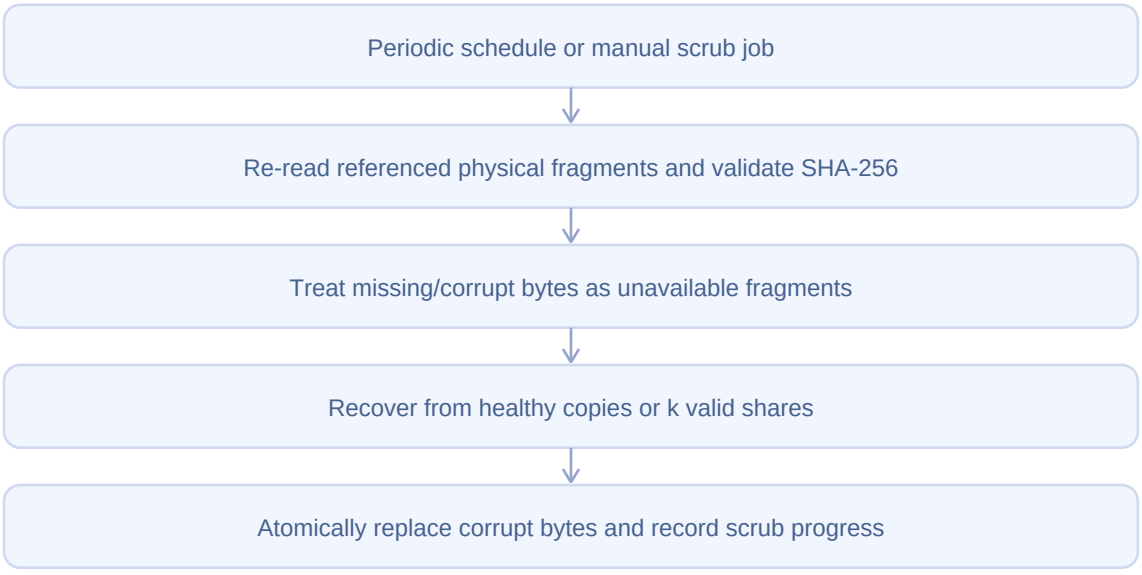


Failure and repair path



Eight nodes provide spare capacity beyond the six distinct nodes used by each 4+2 stripe. Replacements are committed before obsolete physical data becomes eligible for collection.

Background scrubber



Measured comparison

The completed small-file measurement used identical seeded bytes under both policies: 24 logical objects, 22 unique chunks and one duplicate in every ten objects. One physical fragment per unique chunk was removed and rebuilt on the live cluster.

Metric	Replication	Reed-Solomon 4+2
Logical bytes	671,780	671,780
Unique bytes	620,615	620,615
Encoded stored bytes	1,861,845	930,966
Coding overhead	3.0000x	1.5001x
Effective overhead after deduplication	2.7715x	1.3858x
Recovery source bytes	620,615	620,644
Recovery target bytes	620,615	155,161
Rebuilt fragments	22	22

Erasure coding halves coding storage overhead for this configuration. Repairing one erasure fragment reads approximately one original chunk and writes one quarter chunk. Padding explains small differences. Scrub scans and protocol headers are excluded from recovery traffic.

Workload completion

The full generator supports 5,000 small files and ten 100-MiB files per mode. That complete benchmark did not finish before the presentation deadline. Two replicated 100-MiB objects were uploaded and checksum-verified; their repair measurement was interrupted by a concurrent failure test. Do not describe this as a completed full benchmark.

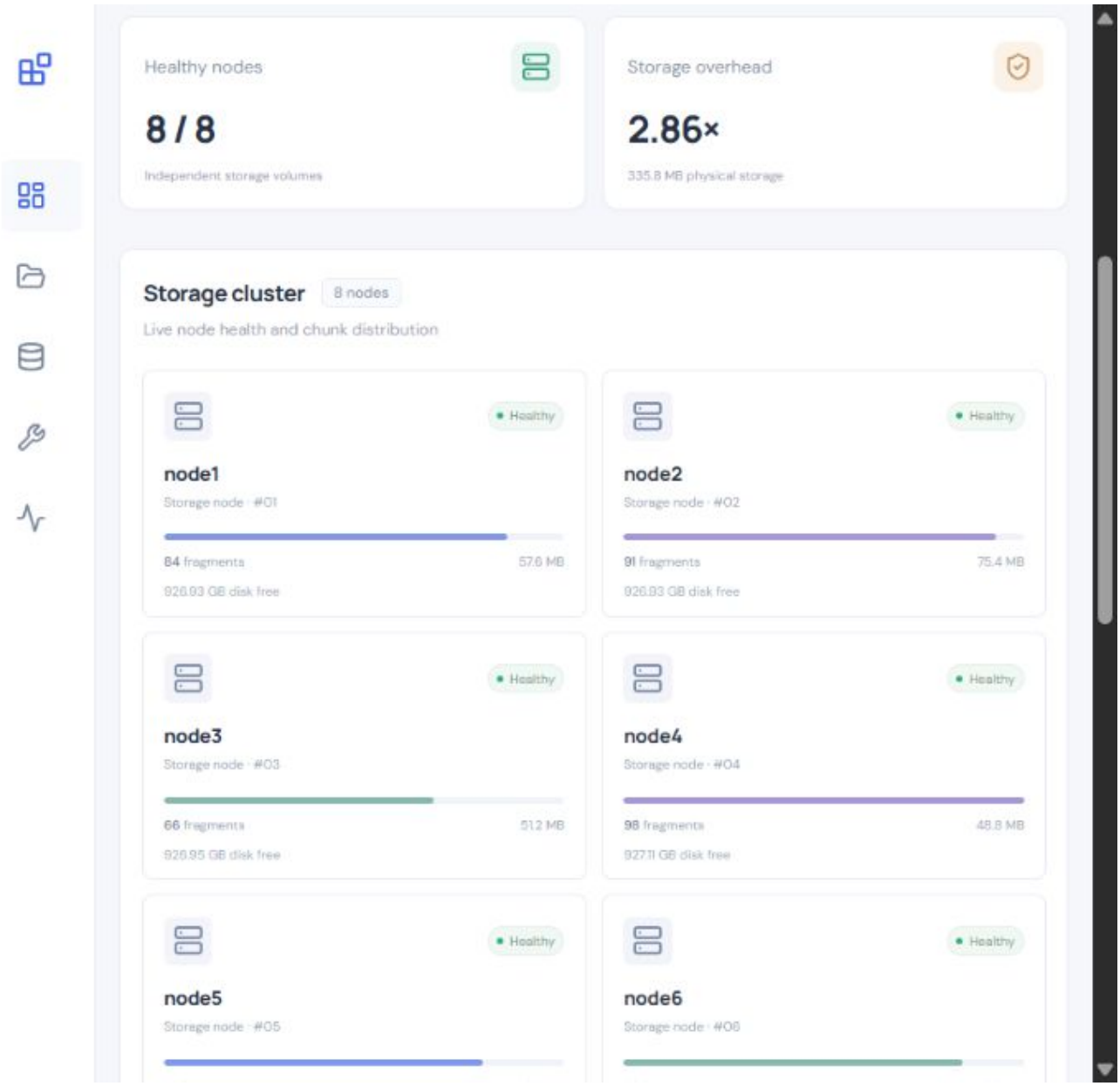
API tests

Recorded JUnit evidence: 10 passing test cases out of 10 recorded cases. Exact output: docs/api-results.xml.

Checks include streaming, cross-chunk and suffix byte ranges, invalid ranges, deduplication, versions, concurrent writes, compression, all 15 possible four-of-six erasure recoveries, authentication and shared rate limiting.

Live cluster overview

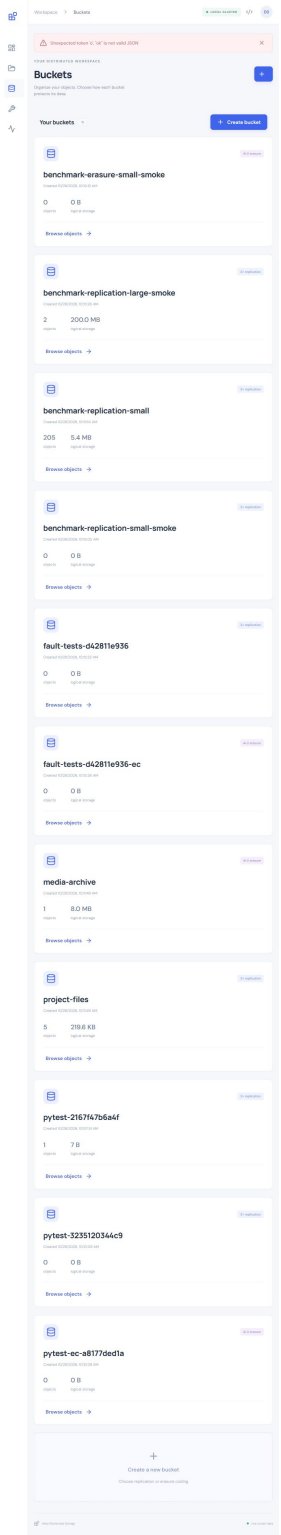
Live browser screenshot showing the node-distribution interface and eight-node health summary during verification.



Recorded screenshot: docs/screenshots/01-live-overview.jpg

Bucket policies

The UI offers independently configured replication and erasure-coding buckets. Some screenshots may record an intermediate restart state; use current live health and the acceptance notes when presenting.



Recorded screenshot: docs/screenshots/02-bucket-policies.jpg

Demonstration checklist

- Start Docker Desktop and the required stack. Open localhost:8088 and use API_KEY from .env.

- Create one replicated bucket and one 4+2 erasure-coded bucket.

- Upload a file; inspect its SHA-256, version, tags and physical fragment locations.

- Download the object and compare its checksum. Demonstrate a byte-range request in Swagger or curl.

- Upload identical bytes under another key and inspect the shared chunk reference count.

- Demonstrate prefix/size/date/tag search after Elasticsearch is healthy.

- Queue a scrub and inspect Maintenance job progress and recovery byte counters.

- Run failure checks separately from benchmark fragment removal, then capture the successful final results.

Source delivery

framework/: integrated services. ui/: React source. docker/: service Dockerfiles. compose.yaml: local topology. tests/: API and coding tests. scripts/: setup, generators, benchmarks and fault checks. deploy/: Kubernetes, Nginx and monitoring. .github/workflows/: CI and Kubernetes deployment. docs/: evidence and run guide.

Practical limits

This lab uses an API key with workspace-wide access and internal service isolation. Transport TLS and multi-user authorization are not configured. The local machine experienced memory exhaustion during final integration, so bonus containers were deprioritized. Remote deployment requires a configured cluster/runner and credentials; GitHub alone does not host Kubernetes.